Written by: ***Konrad Mocarski***

1 Introduction

In this report I present the whole explanations, performance tests and various measurements comparisons made for project Museum Heist (PG-2) written in Python with NumPy and NetworkX, using Gymnasium for environment modeling, and matplotlib with seaborn for plotting the final results. Project was made as a part of Reinforcement Learning course at University of Milan. All computations were performed on the M1 Pro using JetBrains DataSpell for development. Experiments included in the comparisons were performed on (almost always) randomly selected start and target positions in the Museum using NumPy with a lot of hyperparameters tweaking.

The stated problem models a pursuit-evasion scenario within a grid-based museum environment. The adversary is a **stateful thief that uses shortest-path logic** aided by a hacker. Hacker provides active surveillance data to help the thief navigate toward a target painting position and return safely to starting point to escape. On the other hand, the agent is a strictly **stateless security camera** tasked with learning a stochastic policy to intercept the thief (there is no exactly random scenario, where the agent will catch the thief with 100%, therefore the agent must learn which rooms are the most important in given museum layout to maximize thief catching). To achieve this, the agent is trained using the **REINFORCE algorithm** (Monte Carlo Policy Gradient) parameterized by a **Softmax policy**. The learning process may introduce a return baseline for variance reduction, alongside extensive hyperparameter tuning, to ensure stable convergence and catching maximization across randomized starting and target positions.

2 Implementation details

The environment and simulation mechanics are modeled using the Gymnasium API, utilizing NetworkX for dynamic graph-based path-finding, while core numerical computations are driven by NumPy. Final results, learning curves, node importance (action probability distribution) are visualized using Matplotlib and Seaborn.

2.1 Environment selection

We model our Museum Environment as custom environment using the **Gymnasium** framework. This allows us to cleanly and easily separate the stateless nature of the surveillance Agent from a little bit more complex, stateful mechanics of our Adversary (Thief) and the Museum state – starting/painting position selection. Environment itself needs to track **starting/painting position**, **whether thief has obtained the painting** and **`n_counts`**

(which represents for i-th room, when was this room last selected by the surveillance algotrithm)} to run the simulation, allow the Thief to move accordingly and provide rewards.

The agent is by design stateless, so we model the environment with **observation space** of `spaces.Discrete(1)` which always returns 0. As there is not one simple answer to always catching the thief the agent must learn the stationary probability distribution over the map that works in general case based on the museum topology and thief's movement on that topology. This perfectly motivates the use of a parameterized stochastic policy (REINFORCE with Softmax), as deterministic policies would easily be bypassed very easily by the adversary.

The **action space** is modeled as `spaces.Discrete(N)`, where N is the total number of valid rooms in the museum graph. At each round, the agent must focus its single unit of attention to exactly one room. While the action space is pretty simple, the consequences of the actions are a little bit more complex, as the environment tracks a dynamic mathematical cost for the thief to enter a room $1 + \beta \cdot 2^{-(n-1)}$, the agent's action directly affects the graph weights and thief's actions. The agent is not just trying to guess where the thief is, but it is actively manipulating the museum's topology to trap the thief.

The difficulty of this environment scales significantly due to the asymmetric capabilities of the adversary. While the agent is stateless and blind, the thief has perfect knowledge of the map, the painting position, and the exit. The adversary does not take random actions, but mathematically optimizes the path to the target room by weighted shortest path calculations at every round. Additional difficulty for the agent arises by the Hacker intervention – by this assumption, we temporarily remove the node from the map, that agent currently is surveilling. This means the agent cannot simply "camp" the target room; doing so will make a thief wait in current room until the camera changes position.

Environment was designed as this, because it creates a nice optimization landscape. The agent must implicitly learn the topology of a 2D grid identifying chokepoints and corridors, while simultaneously maintaining enough randomness to remain unpredictable to a mathematically optimal adversary.

2.2 Environment modeling & museum topology

To accurately simulate the pursuit-evasion dynamics, the museum is modeled as a discrete $n \cdot m$ 2D grid-world, where each valid cell in the grid represents an accessible room, and the edges between adjacent cells represent connecting doors. Each room (graph vertex) is connected to at most 4 other neighboring vertices.

The environment allows to parse custom maps with walls inside it (museum not necessarily have $n \cdot m$ rooms). Additionally, we've included the possibility to allow different types of starting/painting position choosing. The idea of just **random** sampling of target room positions has been extended to also allow **fixed** positioning and **divided** positioning (in which, we split the museum to left-right parts and randomly choose starting position always on the left side and painting position always on the right side).

In topology representation, graph is treated as undirected, just for extracting nodes and edges. In the adversary logic, the graph is going to be treated as directed, since transfer from i-th room to j-th room can be different, than the other way around, since they might have different `n_counts` values.

2.3 Adversary (Thief)

To create a challenge for the surveillance agent, the adversary in this environment (the thief) is modeled as state-aware agent utilizing deterministic shortest-path logic. Unlike the security camera, the thief has access to the full layout of the museum, the location of the painting, and the exit. On top of that, he is in continuous contact with a hacker, that provides him with the information about currently surveilled room. When a hacker communicates to the thief the information about currently surveilled room, that room is excluded in that round from dynamic path-finding algorithm.

The thief's movement is driven by a dynamic path-finding algorithm evaluating the safest route on directed graph to its current objective (either reaching the painting or returning to the exit). The weight to enter a given room v is calculated using the following decay function: $C(v) = 1 + \beta \cdot 2^{-(n_v - 1)}$, where β is a scaling hyperparameter and n_v is the number of rounds since room v was last checked by the camera.

Because the camera acts as a temporary, impassable wall, it can occasionally cut off the only viable path to the thief's target. When the shortest-path algorithm fails to find a route, the thief falls back to a stalling strategy, remaining in its current room. This prevents the thief from blindly walking into a surveilled room, forcing the camera agent to either anticipate the thief's waiting behavior or systematically clear the surrounding chokepoints.

2.4 Agent (Security Camera)

In contrast to the state-aware adversary, the agent is modeled strictly stateless. Its objective is to learn a generalized, stochastic surveillance policy that maximizes the probability of intercepting the thief, relying entirely on delayed reward signals.

To enforce its stateless nature, the agent is blinded to the environment's underlying Markov Decision Process (MDP). The **observation space** is, that regardless of where the thief is, whether the painting has been stolen, or what room was previously checked, the environment always returns a dummy observation of 0. The **action space** is modeled, that at each round, the agent must allocate its attention to a single room. Therefore, the action space is modeled as discrete space of N actions, where N represents the total number of navigable rooms in the museum topology.

Because the camera lacks continuous tracking capabilities, it relies on episodic rewards to evaluate its performance. Reward for the agent is:

- +1.0 awarded immediately if the camera's selected room exactly matches the thief's current position, terminating the episode as a win.
- −1.0 penalized if the thief successfully secures the painting and returns to the starting exit point undetected, terminating the episode as a loss.
- 0.0 returned for every standard step where the thief is neither caught nor escapes.

The Agent's learning is represented by a parameter vector θ, where each room (action) is assigned a single parameter initialized to zero. To convert (learn) these raw parameters into a valid probability distribution, we use the **Softmax function** with a **temperature parameter** τ:

$$\pi_\theta(a) = Softmax(\frac{\theta_a}{\tau}) = \frac{exp(\frac{\theta_a}{\tau})}{\Sigma_{a' \in \mathcal{A}} exp(\frac{\theta'_a}{\tau})}.$$

Each episode, the agent's updates its parameters via **Gradient Ascent** in REINFORCE method:

$$\theta_{k+1} = \theta_k + \alpha \cdot \nabla_\theta J(\theta_k) = \theta_k + \alpha \cdot \nabla_\theta \, log \, \pi_\theta(a) \, R(\tau).$$

where τ parameter is a **scalar temperature** that scales the logits, that prevents the policy from collapsing onto a single room, just because at some point, somewhere the Agent got lucky and caught the Thief; γ is the **discount factor**, which enforces that the action taken at moment i-th is a little more valuable, than previous actions that led to that given action; and α is a **learning rate**, that controls how much the Agent adjusts its internal parameters in response to estimated errors during training. On top of that we also introduce the concept of **variance reduction (baseline)**, which, when enabled, subtracts the mean return of the episode from each step's return. This centers the rewards—penalizing below-average actions and rewarding above-average ones, which might allow us to converge faster and more stable.

Traditional value-based RL algorithms fail in this environment because there are no distinct states to map action-values to. If the agent attempted to learn a deterministic policy (e.g., always checking the highest-value room), the stateful thief would easily exploit this predictable behavior by routing around the camera's permanent blind spots. Therefore, we make the agent learn an optimal stationary probability distribution by utilizing a Softmax-parameterized Policy Gradient method (REINFORCE). The agent learns to distribute its probability across the museum's critical chokepoints. This ensures the camera behaves stochastically and remains entirely unpredictable to the adversary, while still mathematically favoring the rooms most heavily trafficked by the thief

3 Agent's training and performance evaluation

Because REINFORCE is a Monte Carlo method, the agent must experience an entire episode from start to finish before it can learn anything. Inside the learning loop, the Agent chooses the action to perform (room to surveil) and reports it to the environment until the episode has terminated. We actively track the amount of wins the Agent has achieved during training process to monitor whether he's actually learning something.

Once the episode terminates (either the thief is caught or escapes), the inner loop breaks. We then pass the complete trajectory list to the `agent.update()` method, where the Agent calculates the discounted returns for that specific heist and adjusts its parameter vector θ via **Gradient Ascent** to make the successful actions more likely in the future.

We tweak different hyperparameters (establishing a **custom museum topology** and **layout of starting and painting positions**, choosing the **number of training episodes**, a **discount factor** γ, an Adversary **"prudence" constant** β, a **Softmax temperature** τ, a **learning rate** α and setting whether to use **variance reduction** or not) to compare learning curves and final probability distribution.

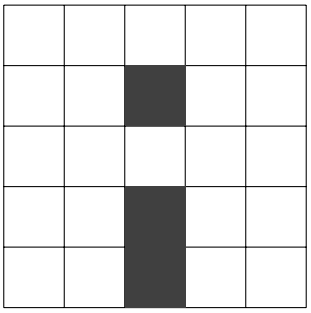
We evaluate the agent on the same hyperparameters, as he has learned on, but we still choose start/painting positions as (almost always) random, so that these are heists that the

agent has "probably" not seen before. We perform different experiments and comparisons in the next chapter.

4 Various experiments comparisons

We train our agent on 5000 episodes. This amount turned out to be pretty reasonable and stable throughout development process. On that amount, the agent stabilizes well informs us, that he didn't just got lucky in previous episodes, but keeps his performance throughout that whole time. We later evaluate the agent on 1000 episodes.

To maintain the reliability of the experiments (tweaking hyperparameters), we will mostly train and evaluate our agent on the museum topology given as this grid:



Rysunek 1: Custom 5x5 Museum Topology. Dark cells represent impassable walls.

Later, as out of curiosity, we will add and explore a few additional examples with less walls or different topologies, but without tweaking the hyperparameters as much.

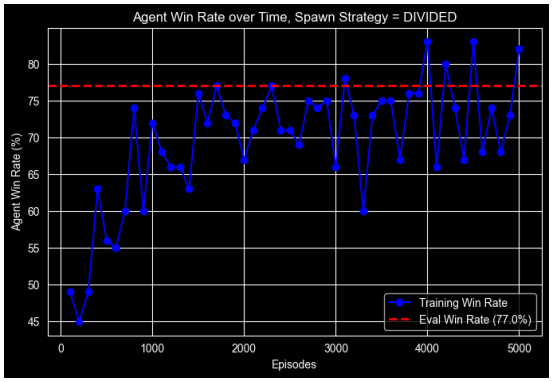
mozna tez poodpalac duzo roznych eksperymentow dla roznych parametrow i np. jak bedziemy zmieniac gamme i to nic nie da (ani lepiej ani gorzej) to na sam koniec po prostu napisac ze tweakowanie parametrow x, y, z, ... nic nie dalo / nic nie zmienilo to oszczedzi sporo miejsca i czasu

4.1 DIVIDED Spawn Strategy

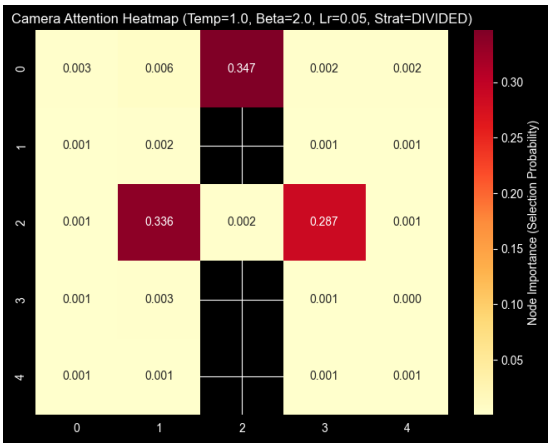
We "split" the museum map into left-right parts. The thief spawns always on the left side and the painting spawns always on the right side of t he museum.

4.1.1 Without baseline

We notice that on the initial setup with $\beta = 2.0$ and $\tau = 1.0$ the model performs pretty decent averaging pretty much all the time the final evaluation score at around 75%:



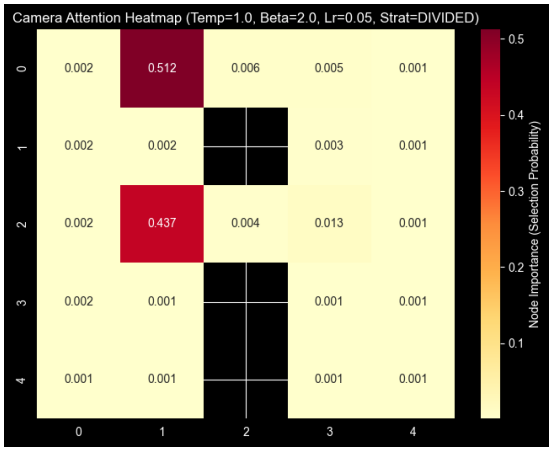
(a) Learning curve



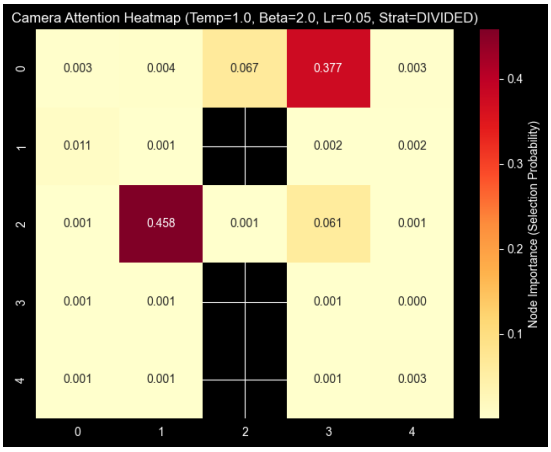
(b) Node importance

Rysunek 2: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

Of course, since the spawn and painting position are every time chosen by random, the learning curve can have sometimes variate and achieve less win ratio. Nevertheless, overall looking at the node importance distribution, we see that the agent identifies pretty well the chokepoints on the museum layout – identifing the best positions to surveil to be able to catch the thief most of the times. The experiments are reproducible, with obviously some variations regarding chokepoints, but the variations are related rather to specific rooms close to chokepoints, rather than completely different unrelated rooms in the museum.



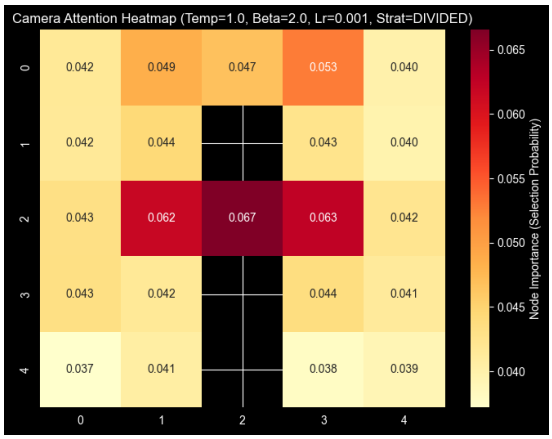
(a) Node importance



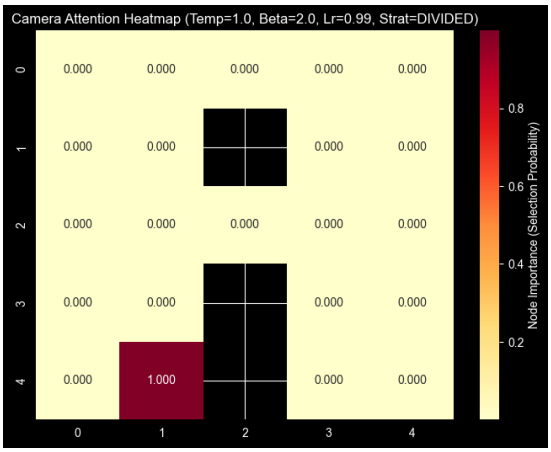
(b) Node importance

Rysunek 3: A little bit different node importances for the same setup

After running a few iterations for learning with different γ value, thus we will not investigate it here any further. Running for different α parameter, we tend to observe, that obviously (since it's learning rate), running for super high values makes our model highly overfit and actually learn nothing and running for super low values, makes our model learn extremely slow and not that it's any better (even with higher amount of episodes) when it comes to performance, so we will keep in further experiments with our initial value.



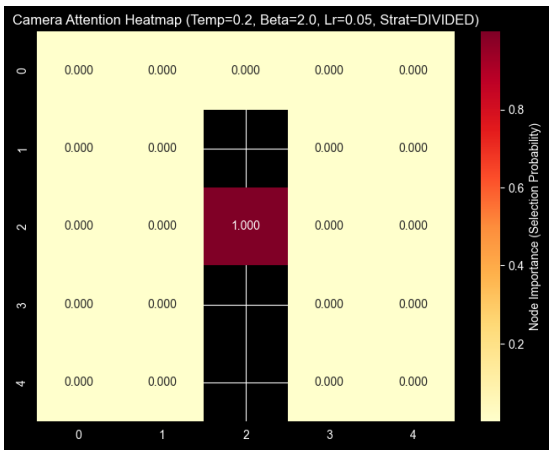
(a) Node importance for $\alpha = 0.001$



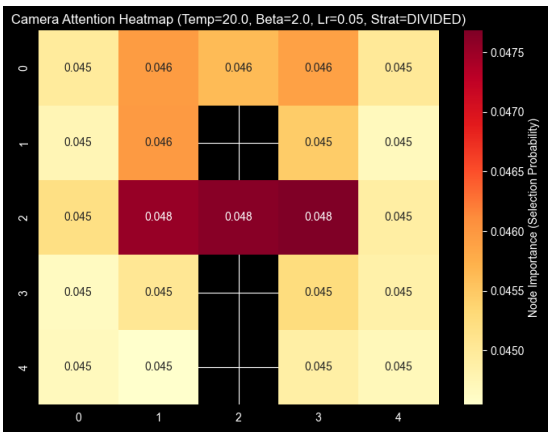
(b) Node importance for $\alpha = 0.99$

Rysunek 4: Node importance for different α

We also notice, that tweaking β parameter for this kind of topology doesn't affect the learning curve nor the node importance distribution. On the other hand, since τ is the **scalar temperature**, for which lower values indicate rather "greedy" approach and they allow the agent to fall into a single room just because he got lucky catching a thief there at some point; for higher values, we allow the agent to keep more exploration, thus we notice, that he doesn't really maximize the winrate, since he tries to explore all the time every room almost equally (and yes, he identifies te rooms pretty well, but for given museum topology and spawn strategy, this rather is not helpful to increate the winrate).



(a) Node importance for $\tau = 0.2$

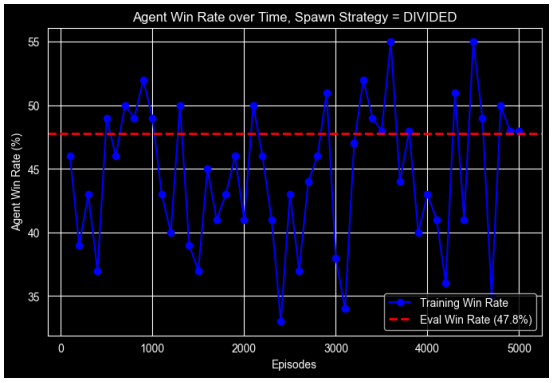


(b) Node importance for $\tau = 20.0$

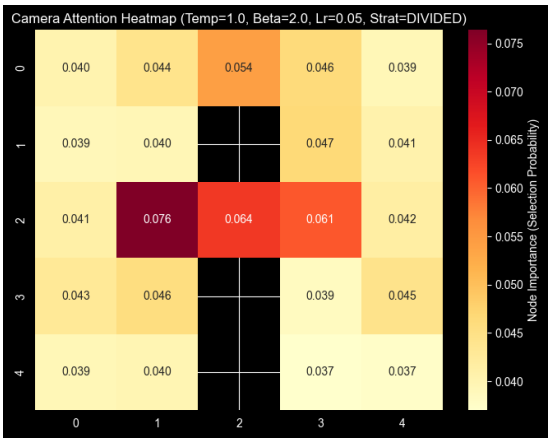
Rysunek 5: Node importance for different τ

4.1.2 With baseline

For running the agent with average baseline for REINFORCE, we notice that, even tho baseline is used for the algorithm to reduce variance by centering the return, the algorithm actually performs worse with the baseline (maybe I screwed up something?). It correctly identifies the importance of nodes, but keeps the importances still more distributed.



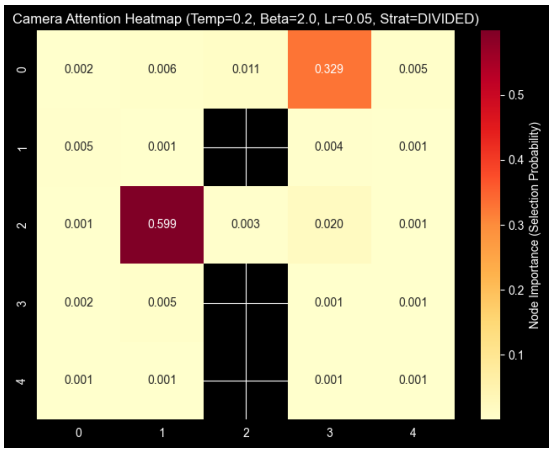
(a) Learning curve with baseline



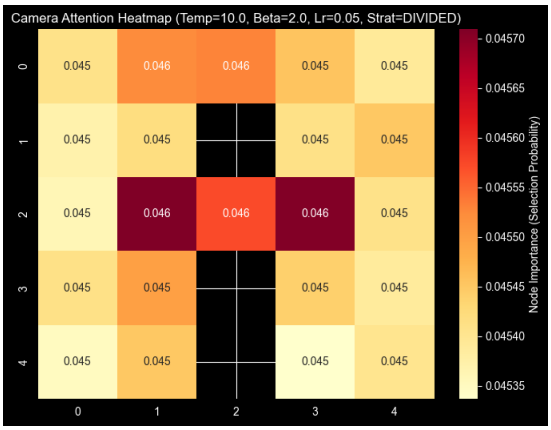
(b) Node importance with baseline

Rysunek 6: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

Again tweaking the β parameter doesn't give us much new information and winratio increase, but τ has a bigger impact here. Lower values, which indicates more "greedy" approaches, tend to learn pretty well, since τ enforces greediness and baseline enforces variance reduction, the result comes out pretty nice. On the other hand, when setting higher τ, which encourages the exploration we tend to notice, that every node gets the exactly same amount of importance.



(a) Node importance for $\tau = 0.2$ with baseline



(b) Node importance for $\tau = 20.0$

Rysunek 7: Node importance for different τ with baseline

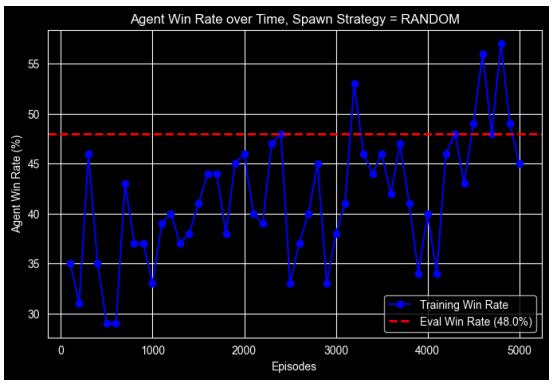
4.2 RANDOM Spawn Strategy

We sample at random the position of the thief and position of the painting.

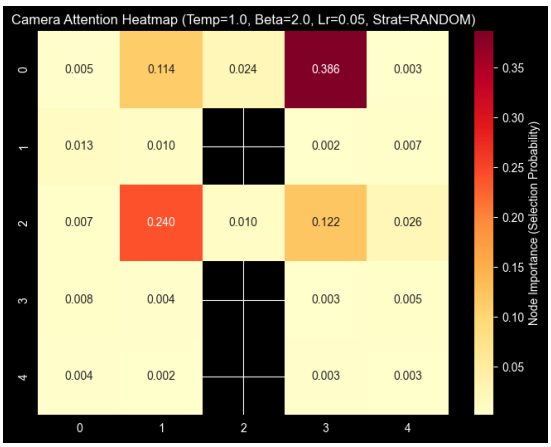
4.2.1 Without baseline

Obviously, here the winratio results are going to be much worse, since the starting and target positions are random across the whole map, thief doesn't have to cross the museum

bottlenecks to get to the painting and go back. Nevertheless, we tend to notice, that for given museum topology agent identifies the chokepoints pretty well anyway.



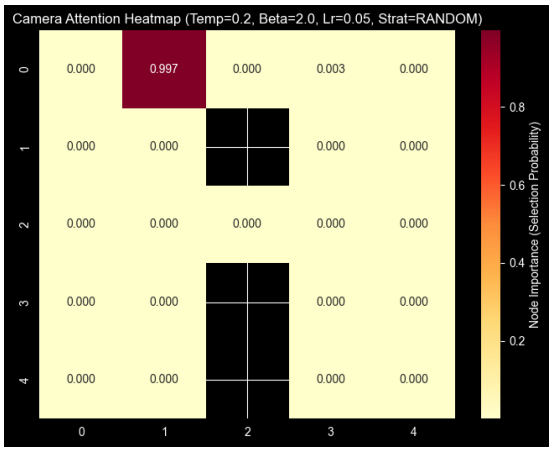
(a) Learning curve



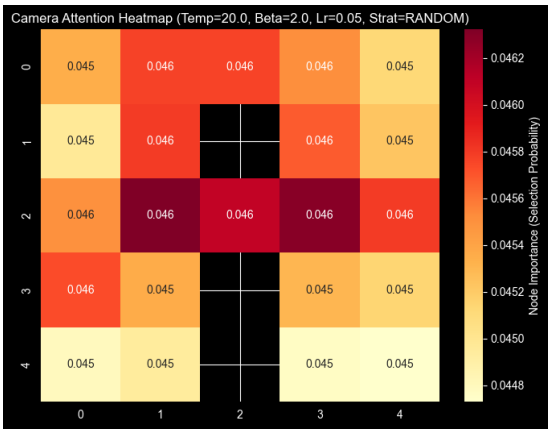
(b) Node importance

Rysunek 8: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

We notice the same relation with τ scalar, as in the previous spawn strategy (which makes sense (at least I think so), as higher values encourages exploration and lower encourages greediness) – anyway for this museum topology, the spawn strategy *DIVIDED* and *RANDOM* are almost similar (but the winrate is worse, since this time we don't enforce the thief to cross museum's bottlenecks).



(a) Node importance for $\tau = 0.2$

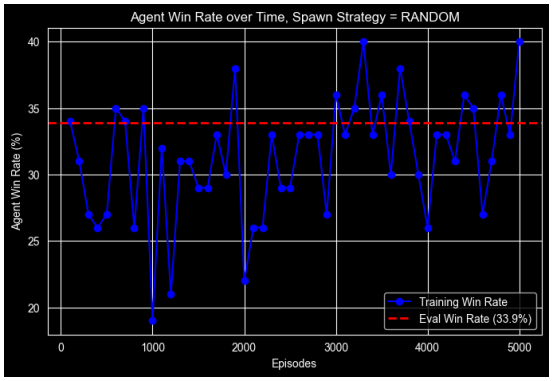


(b) Node importance for $\tau = 20.0$

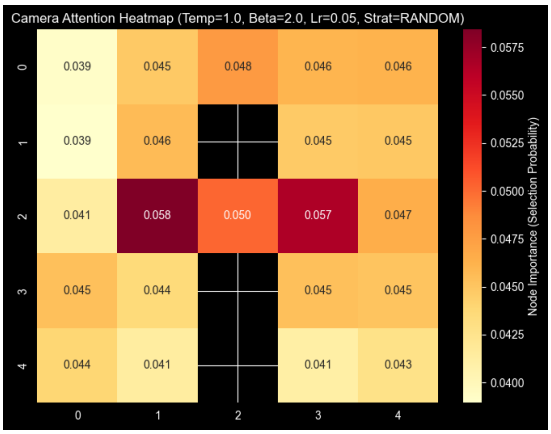
Rysunek 9: Node importance for different τ

4.2.2 With baseline

Again, similarly to *DIVIDED* strategy, but importances are a little bit more distributed, since strategy is random and thief/painting spawns randomly somewhere on the map, which by default enforces the agent to perform more exploration.



(a) Learning curve



(b) Node importance

Rysunek 10: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

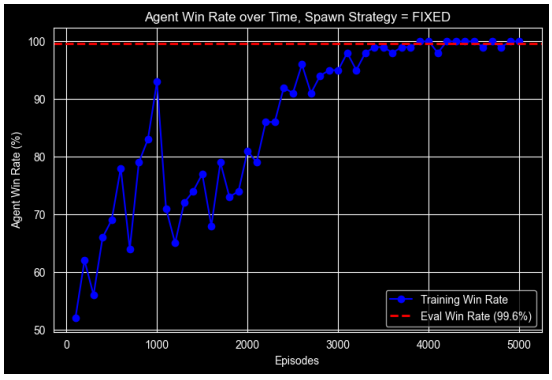
Since the results for tweaking the β and τ parameters were actually pretty same to the results obtained in the *DIVIDED* strategy for spawning museum layout, I will allow myself to omit these plots here.

4.3 FIXED Spawn Strategy

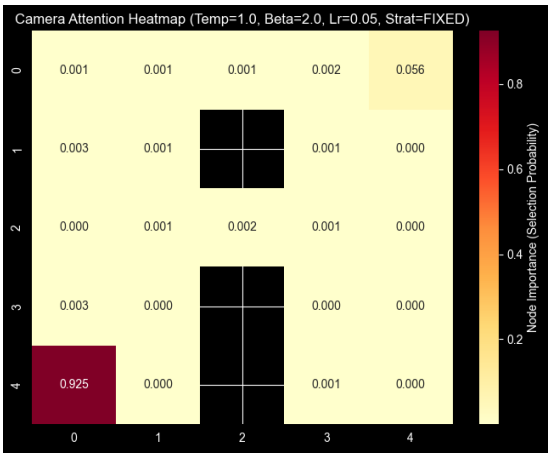
We force the thief to spawn always in the bottom-left corner and the painting in the top-right corner of the museum.

4.3.1 Without baseline

We notice, that in this situation, the results are the most reproducible, as the agent learns that the thief is (or has to reach) always the bottom-left or top-right corner. This results in the learning curve with almost or even exactly 100% winratio certainty and node importances exactly representing the situation.



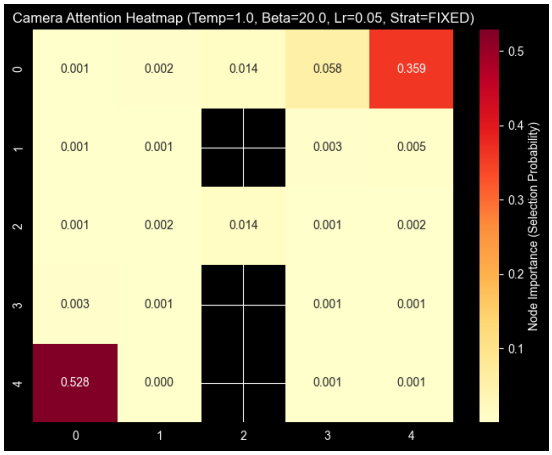
(a) Learning curve



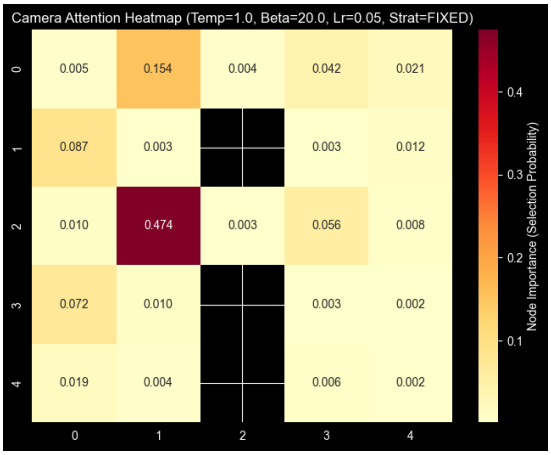
(b) Node importance

Rysunek 11: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

We notice, that when tweaking "prudence" constant for given scenario, we notice, that the node importance distribution can be a little bit more distributed as thief might move more wisely, but the agent still correctly identifies the nodes that he should catch the thief.



(a) Node importance



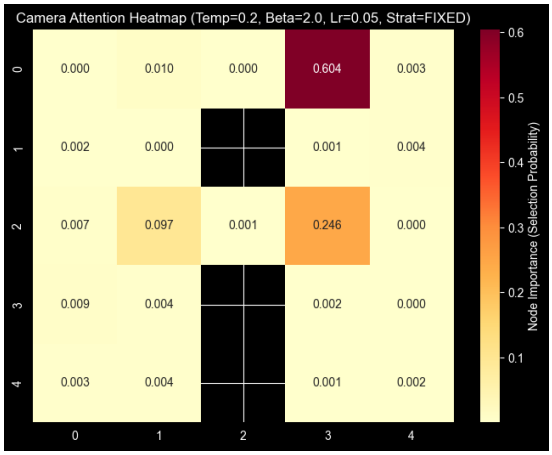
(b) Node importance

Rysunek 12: Node importance for $\beta = 20.0$

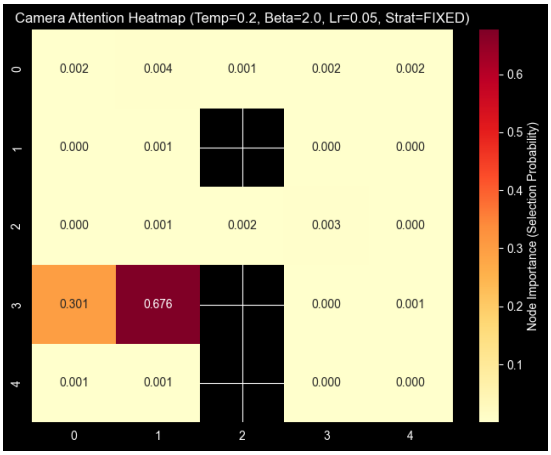
Results for tweaking the τ parameters were pretty same to the results obtained in the DIVIDED strategy for spawning museum layout, I will allow myself to omit these plots here. What is actually fun to watch, that if we set $\tau = 0.2$ and enforce more "greedy" policy, when the agent catches the thief at the starting bottom-left position, obviously we will obtain the 100% winratio in the end, but this metric is not very sufficient, but rather a fun fact.

4.3.2 With baseline

Applying baseline and rather low/"greedy" temperature value makes the agent to correctly identify the museum chokepoints (and yes the winrate in not 100%, since the agent stops looking at just the starting point, but I think this results is rather more interesting, since the idea of theft prevention is identifying museum's chokepoints and not just looking at the entrance).



(a) Node importance



(b) Node importance

Rysunek 13: Different results for Node importance for $\tau = 0.2$ with baseline

Other than that, we notice nothing new in the results, so please allow me not to paste the same charts over and over again.

5 Conclusions, limitations and additional information

I hope this report was a least a little bit interesting and fun to read, I've spent on it probably more time than on actually coding/thinking. Sorry that it made a little bit more than 10 pages, but squeezing the pictures/plots inside the .tex pdf is kind of hard to make them still readable without enforcing you to zoom in the pdf as much as possible.

Overall this task was super fun to write and learn from it. We notice that the best agent performance was achieved by not tweaking the hyperparameters that super much, and making them stay rather close to each other. The museum topology is super important parameter here (thief/painting positions, as well as, the museum's bottlenecks), since we notice that when positions are sampled by random, the node importance is more distributed, as the agent doesn't know that much what is exactly going on (which is a hard limitation to training the agent). When we enforce one entrance and one painting position, the agent is always pretty almost sure about catching the thief. For the DIVIDED spawn strategy, the agent identified the museum's chokepoints pretty well and I think (when I actually got the idea of it) that was the best metric to evaluate whether the agent is actually leaning something. On top of that as pointed out, higher temperature encourages us to never stop exploring, which leads to not really valuable report about node importances (agents tells us that the thief can be almost everywhere equally probable). On the other hand setting it too low, tells us that agent caught the thief somewhere and awards himself with super big reward for that, which in consequence leads to not learning anything further, which makes actually no sense, as thief then escapes all the time. In the end, I don't really know that the baseline was that helpful here (or maybe I've screwed something up, but the idea was rather pretty simple, so I dunno), so we decide to move forward without applying it to the algorithm.

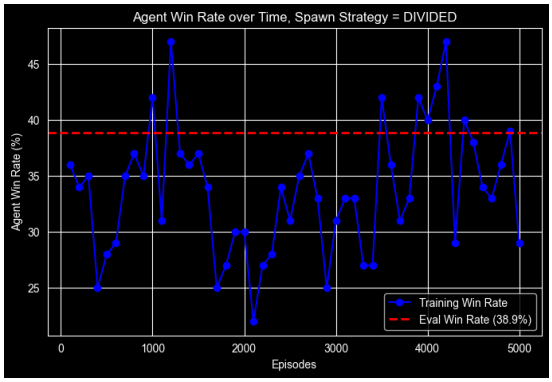
In the appendix, we present the results for additional interesting museum topologies, that didn't make it to the main text, to not make it 20 pages long (even when these are almost always just pictures).

6 Appendix results

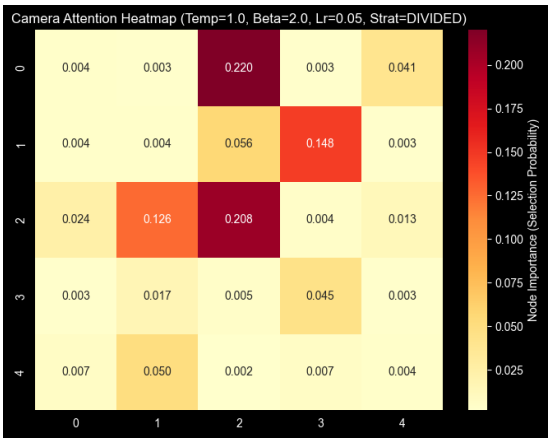
Here we present learning curves and node importance distribution for different type of museum topologies, that I've found interesting and I didn't try to make them squeeze inside the main part of the report. All these below were run for DIVIDED and RANDOM spawning strategy with $\beta = 2.0, \tau = 1.0, \gamma = 0.99, \alpha = 0.05$. Here we will not present the museum topology, as the topology itself will be presented on the node importance heatmap.

6.1 DIVIDED Spawn Strategy

6.2 Without baseline

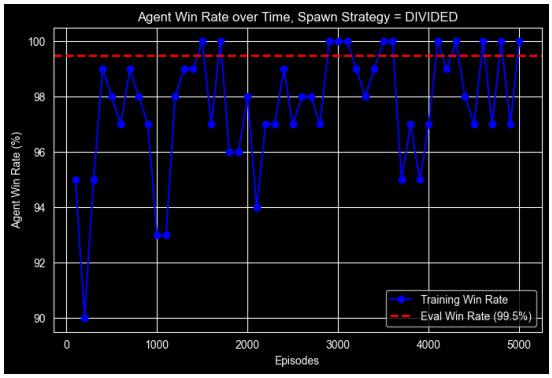


(a) Learning curve

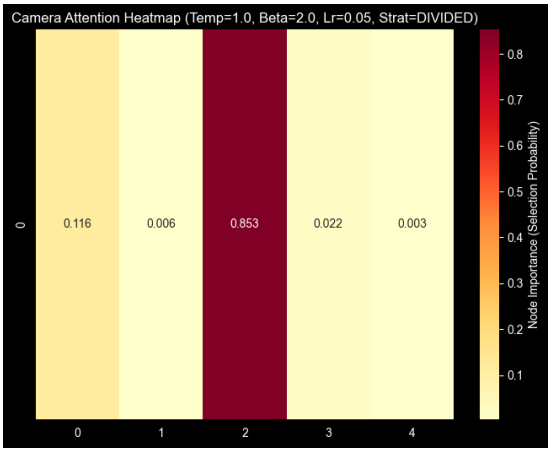


(b) Node importance

Rysunek 14: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

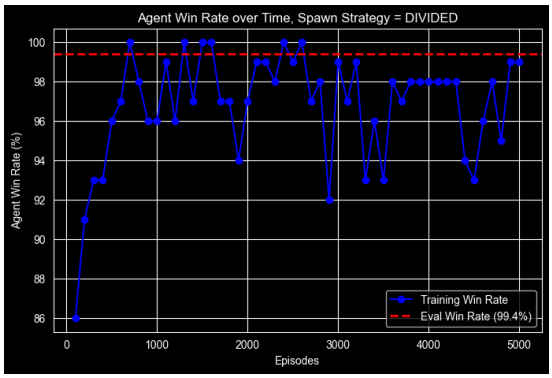


(a) Learning curve

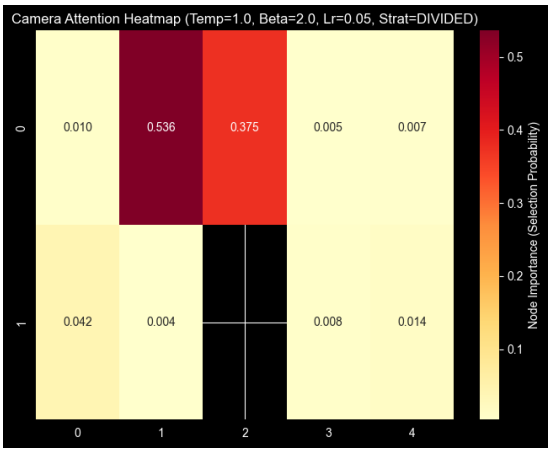


(b) Node importance

Rysunek 15: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$



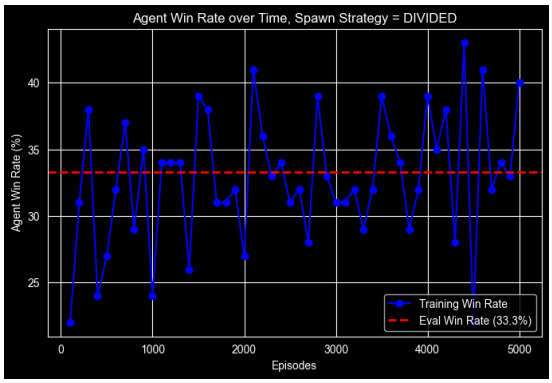
(a) Learning curve



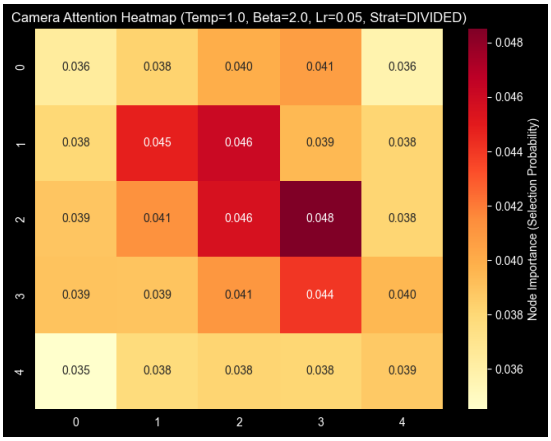
(b) Node importance

Rysunek 16: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

6.3 With baseline

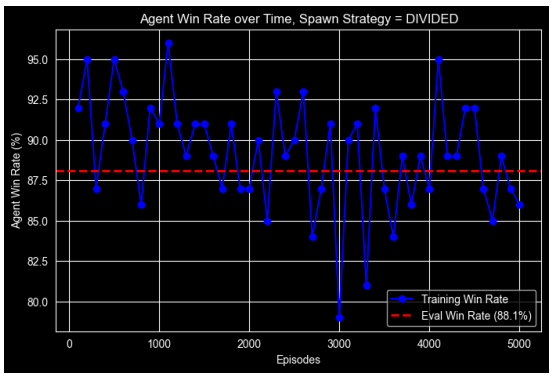


(a) Learning curve

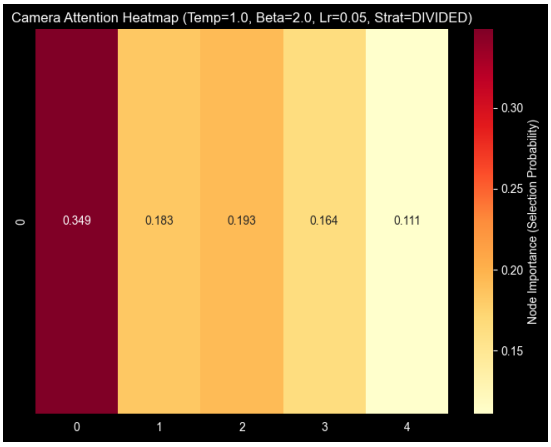


(b) Node importance

Rysunek 17: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

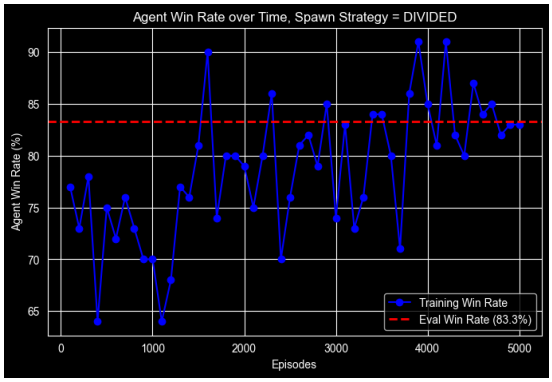


(a) Learning curve

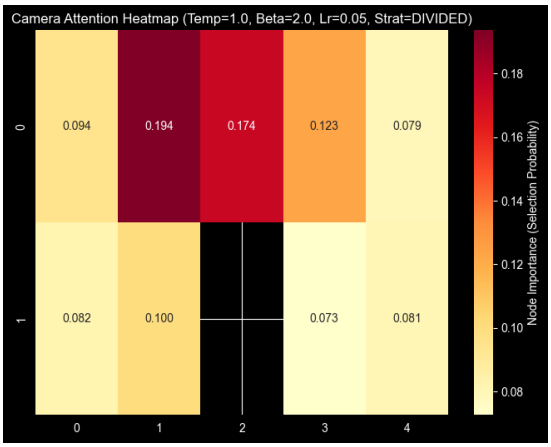


(b) Node importance

Rysunek 18: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$



(a) Learning curve

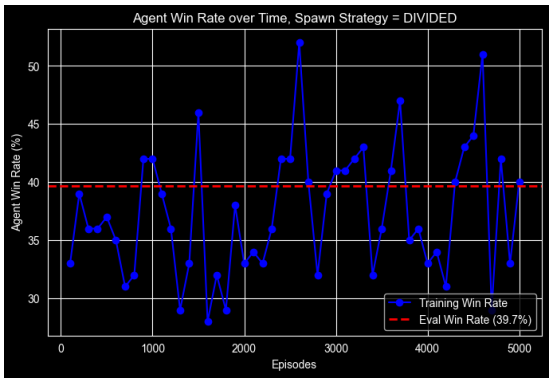


(b) Node importance

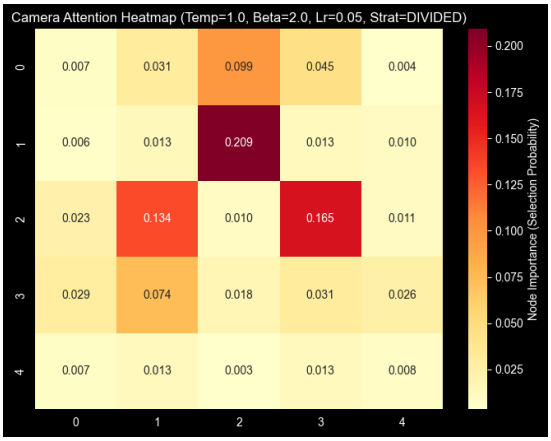
Rysunek 19: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

6.4 RANDOM Spawn Strategy

6.5 Without baseline

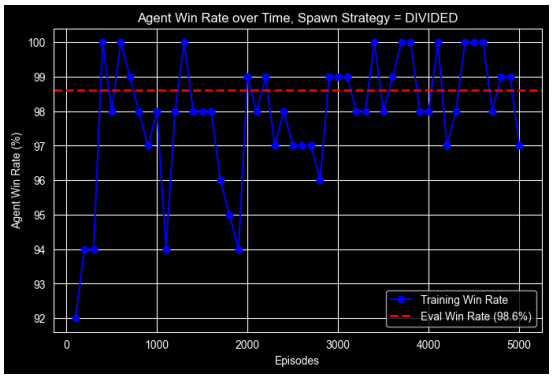


(a) Learning curve

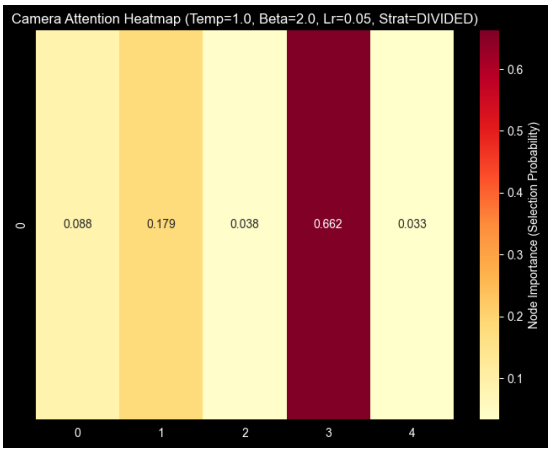


(b) Node importance

Rysunek 20: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

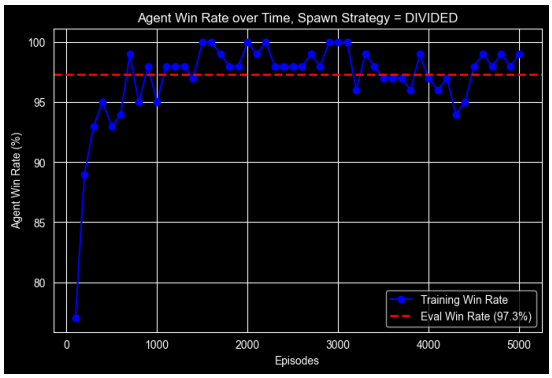


(a) Learning curve

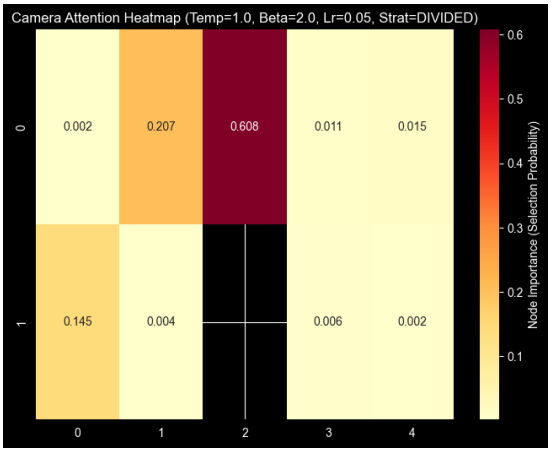


(b) Node importance

Rysunek 21: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$



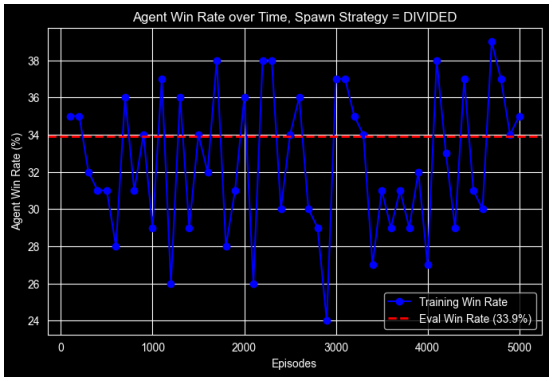
(a) Learning curve



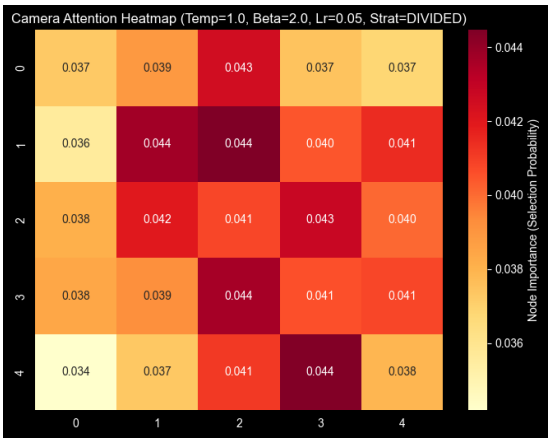
(b) Node importance

Rysunek 22: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

6.6 With baseline

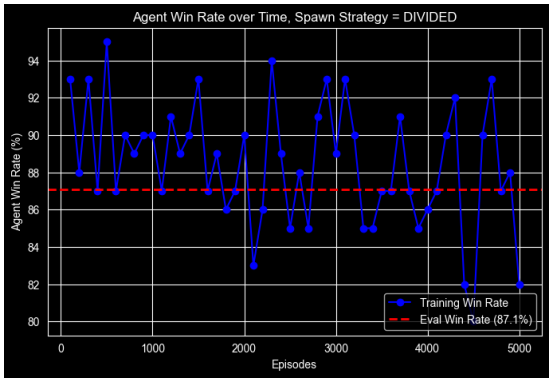


(a) Learning curve

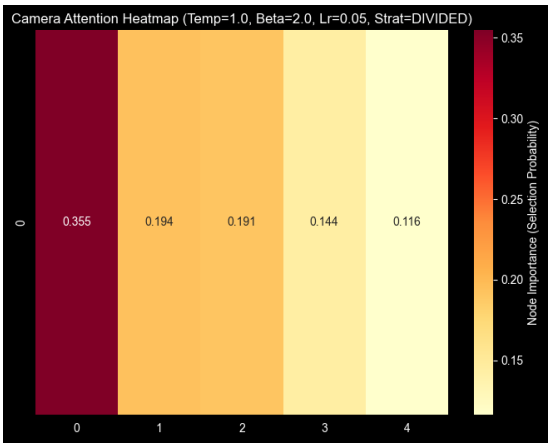


(b) Node importance

Rysunek 23: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

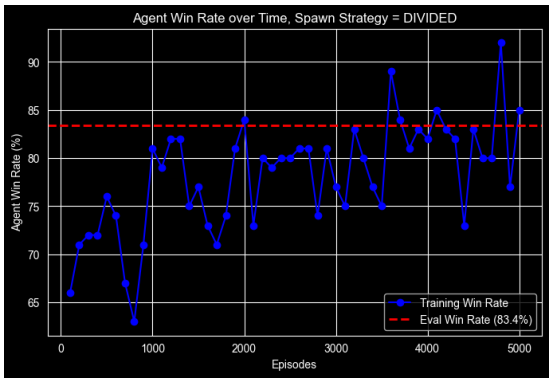


(a) Learning curve

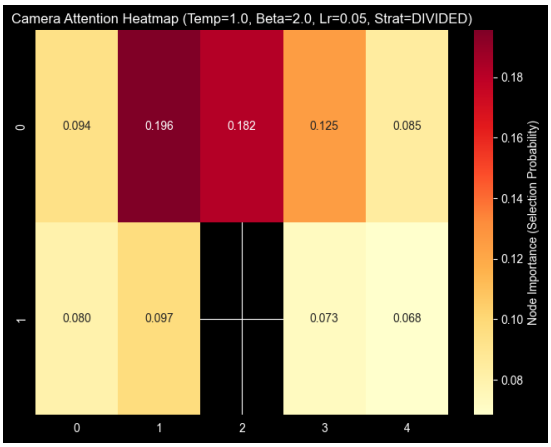


(b) Node importance

Rysunek 24: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$



(a) Learning curve



(b) Node importance

Rysunek 25: Plots for $\beta = 2.0,\ \tau = 1.0,\ \gamma = 0.99,\ \alpha = 0.05$

7 Thanks

Thanks for reading.